





WHO ARE WE?

MAHADIO TLAKA U23545098

DIYA NAROTAM U23533596

CHINMAYI SANTHOSH U24585671

SHAVIR VALLABH U23718146

JAITIN MOODALLY U23621372

AYUSH BEEKUM U23596351

FABIO BERRINO U23526387

WHAT ARE WE:

- The project focuses on designing and implementing a **Greenhouse Nursery Simulator**, a virtual representation of a plant nursery.
- It captures the beauty, complexity, and structure of real-world nursery operations.
- It **models the interaction between plants, customers, and staff** within a dynamic and evolving environment.
- The project aims to apply **object-oriented programming (OOP)** and Gof design patterns.
- The goal is to create a **flexible, maintainable, and scalable system**.
- The simulator demonstrates how design patterns simplify complex problems, including:
 - Managing different plant types
 - Handling unique care requirements
 - Coordinating staff duties
 - Facilitating customer interactions



Functional requirements :

Greenhouse/Garden Area:

- Simulates diverse plant growth and life cycles
- Manages watering, sunlight, pruning & fertilising needs
- Tracks inventory and supports adding new stock

Staff:

- Maintains plant health and monitors growth
- Manages inventory and assists customers
- Performs care tasks like watering and pruning

Customers and Sales Floor:

- Browse and purchase available plants
- Customise orders with pots or packaging
- Interact with staff for advice and assistance

SIMULATION

HOW IT WORKS:

OUR SELF RUNNING SIMULATION AUTONOMOUSLY MANAGES THE GREENHOUSE NURSERY, MONITORING PLANT HEALTH, STAFF ACTIVITY AND CUSTOMER INTERACTIONS TO ENSURE SMOOTH OPERATION.

THE SIMULATION TRACKS KEY METRICS SUCH AS:

PLANT HEALTH – WATER, SUNLIGHT, FERTILIZER AND PRUNING LEVELS

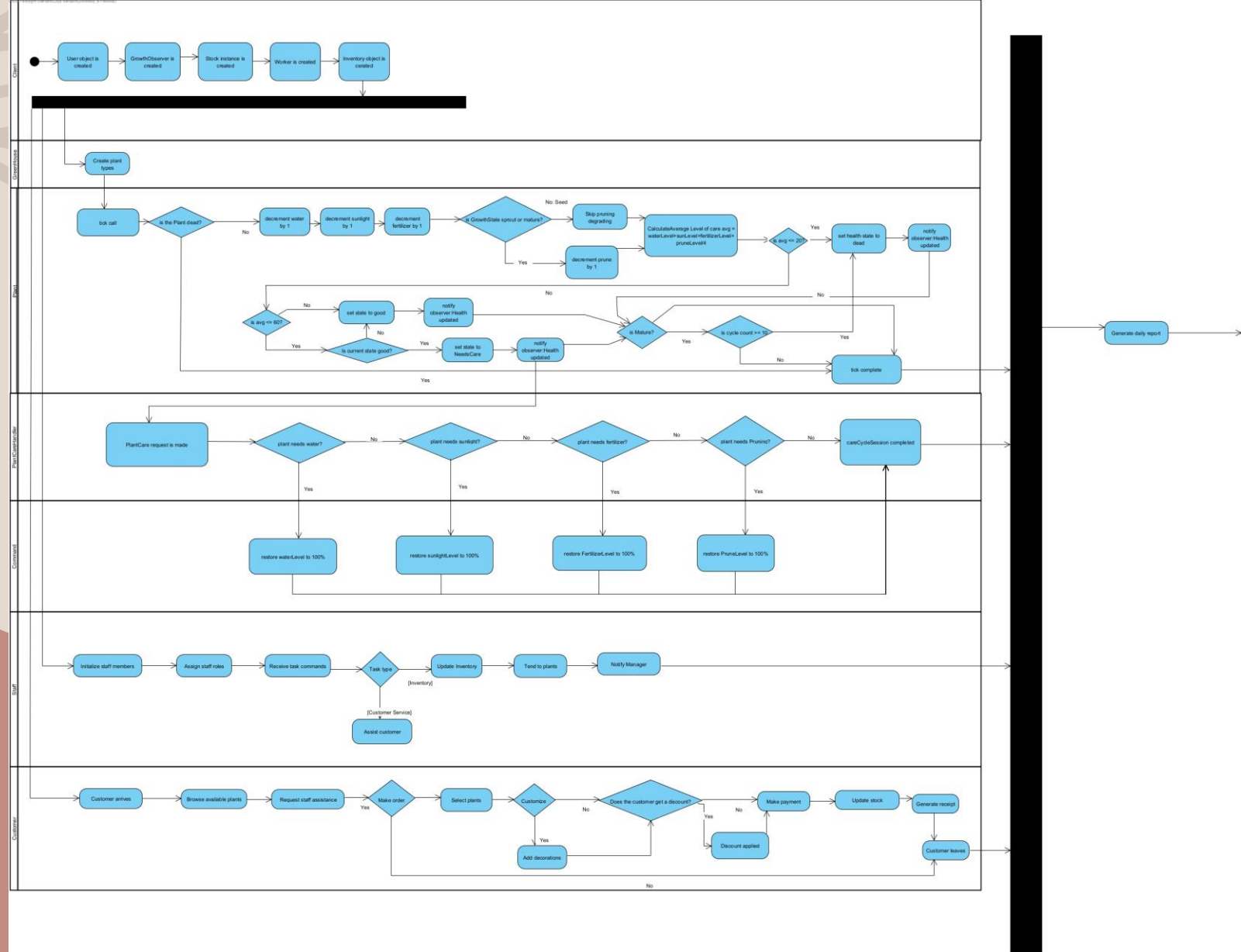
GROWTH PROGRESS – TRACKS EACH PLANTS STAGE FROM SEEDLING TO MATURITY

INVENTORY LEVELS- MONITORS STOCK AVAILABLE FOR SALE OR IN GROWTH

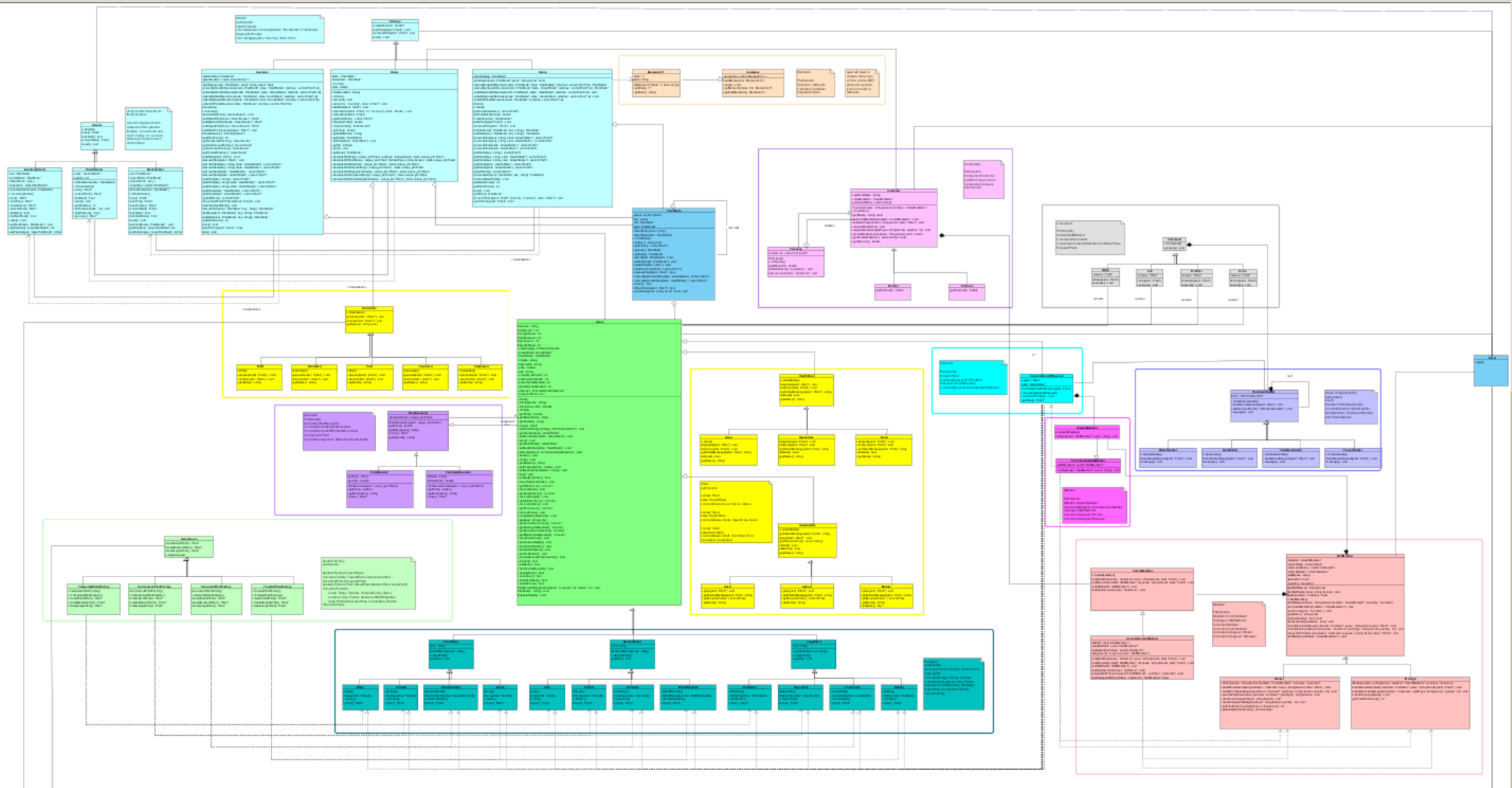
STAFF EFFICIENCY- MEASURES HOW QUICKLY AND EFFECTIVELY STAFF RESPOND TO PLANT NEEDS

CUSTOMER SATISFACTION- REFLECTS HOW WELL CUSTOMER NEEDS ARE MET

Activity Diagram



CLASS DIAGRAM



Abstract Factory & Prototype :

Participants:

Abstract Factory: GreenHouse

Concrete Factory: TropicalPlant, CarnivorousPlant, SucculentPlant, TemperatePlant

Abstract Product: Plant(SmallPlant, MediumPlant, LargePlant)

ConcreteProducts:

Small: Daisy,Sundew,HenAndChick,Nerve

Medium:Lilac,Pitcher,AloeVera,BirdOfParadise

Large:WhiteOak,Nepenthes,Condalabra,Rubber

Client:Inventory

Prototype: Plant and small, medium and large plants

concretePrototype: Daisy, Sundew, HenAndChicks, Nerve, Lilac,

Pitcher, AloeVera, BirdOfParadise , WhiteOak, Nepenthes,

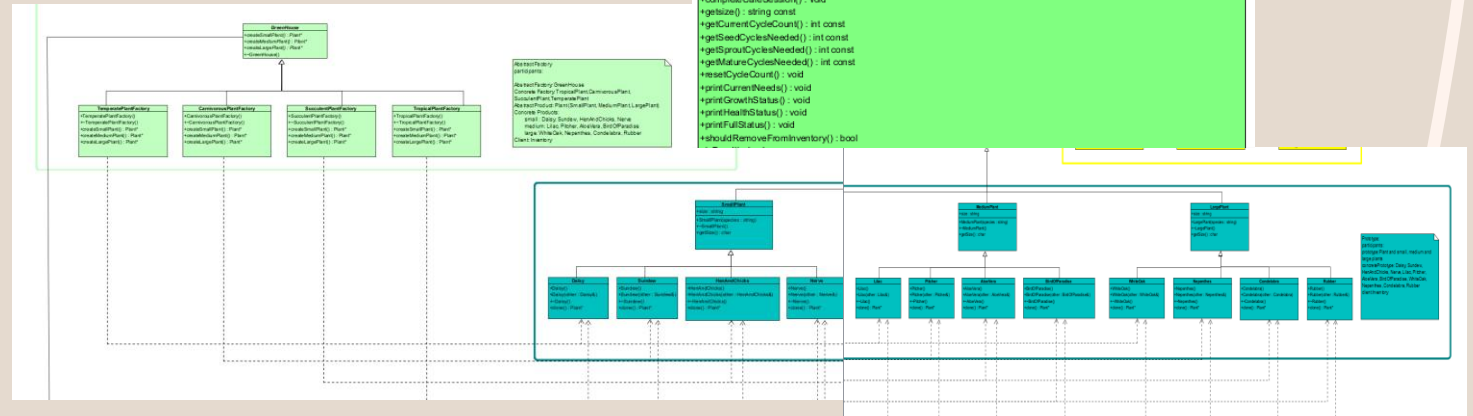
Condalabra, Rubber

Client: Inventory

```
class Plant {
    Species : string
    WaterLevel : int
    SunlightLevel : int
    FertilizerLevel : int
    SproutLevel : int
    GrowthStage : PlantGrowthStage*
    growthState : GrowthState*
    healthState : HealthState*
    climate : string
    description : string
    price : double
    size : string
    currentCycleCount : int
    seedCyclesNeeded : int
    sproutCyclesNeeded : int
    matureCyclesNeeded : int
    observer : ConcreteGrowthObserver*
    readyForStock : bool

    +Plant()
    +Plant(species : string)
    +Plant(const other : Plant&)
    ~+Plant()

    +getPrice() : double
    +getDescription() : string
    +getClimate() : string
    +clone() : Plant*
    +setCareStrategy(strategy : PlantCareHandler*) : void
    +getGrowthState() : GrowthState*
    +setGrowthState(state : GrowthState*) : void
    +grow() : void
    +getHealthState() : HealthState*
    +setHealthState(state : HealthState*) : void
    +attach(observer : ConcreteGrowthObserver*) : void
    +detach() : void
    +notify() : void
    +getSpecies() : string
    +setPrice(newPrice : double) : void
    +setDescription(newDesc : string) : void
    +tick() : void
    +isReadyForStock() : bool
    +markReadyForStock() : void
    +getWaterLevel() : int const
    +restoreWater() : void
    +getSunlightLevel() : int const
    +restoreSunlight() : void
    +getFertilizerLevel() : int const
    +restoreFertilizer() : void
    +getSproutLevel() : int const
    +restorePlant() : void
    +completeCareSession() : void
    +getSize() : string const
    +getCurrentCycleCount() : int const
    +getSeedCyclesNeeded() : int const
    +getSproutCyclesNeeded() : int const
    +getMatureCyclesNeeded() : int const
    +resetCycleCount() : void
    +printCurrentNeeds() : void
    +printGrowthStatus() : void
    +printHealthStatus() : void
    +shouldRemoveFromInventory() : bool
}
```



DECORATOR:

Participants :

Decorator :

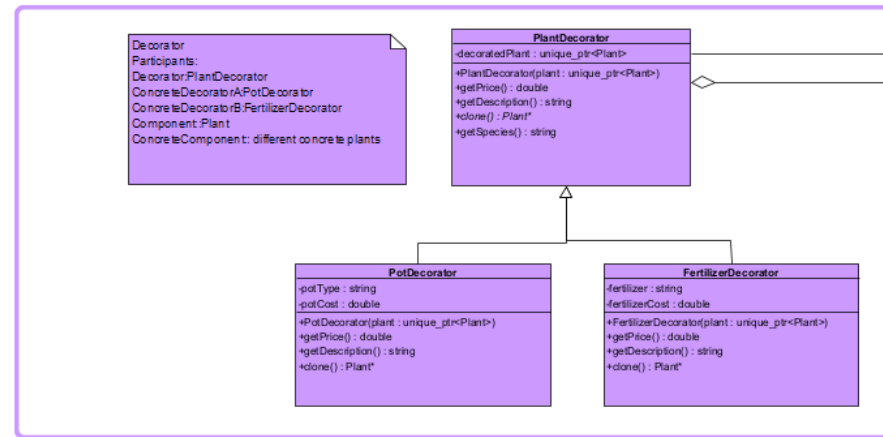
PlantDecorator

ConcreteDecorator :

PotDecorator &

FertilizerDecorator

Component: Plant



```

Plant
#species : string
#waterLevel : int
#sunlightLevel : int
#fertilizedLevel : int
#pruneLevel : int
#growthStage : int
-careStrategy : PlantCareHandler*
-growthState : GrowthState*
-healthState : HealthState*
-climate : string
-description : string
-price : double
-size : string
-currentCycleCount : int
-seedCyclesNeeded : int
-sproutCyclesNeeded : int
-matureCyclesNeeded : int
-observer : ConcreteGrowthObserver*
-readyForStock : bool

+Plant()
+Plant(species : string)
+Plant(const other : Plant&)
~Plant()
+getPrice() : double
+getDescription() : string
+getClimate() : string
+clone() : Plant*
+setCareStrategy(strategy : PlantCareHandler*) : void
+getGrowthState() : GrowthState*
+setGrowthState(state : GrowthState*) : void
+grow() : void
+getHealthState() : HealthState*
+setHealthState(state : HealthState*) : void
+attach(observer : ConcreteGrowthObserver*) : void
+detach() : void
+notify() : void
+getSpecies() : string
+setPrice(newPrice : double) : void
+setDescription(newDesc : string) : void
+tick() : void
+isReadyForStock() : bool
+markReadyForStock() : void
+getWaterLevel() : int const
+restoreWater() : void
+getSunlightLevel() : int const
+restoreSunlight() : void
+getFertilizerLevel() : int const
+restoreFertilizer() : void
+getPruneLevel() : int const
+restorePrune() : void
+completeCareSession() : void
+getSize() : string const
+getCurrentCycleCount() : int const
+getSeedCyclesNeeded() : int const
+getSproutCyclesNeeded() : int const
+getMatureCyclesNeeded() : int const
+resetCycleCount() : void
+printCurrentNeeds() : void
+printGrowthStatus() : void
+printHealthStatus() : void
+printFertilStatus() : void
+shouldRemoveFromInventory() : bool
    
```

State

Participants:

Context: Plant

state: GrowthState

concreteStates: Seed,

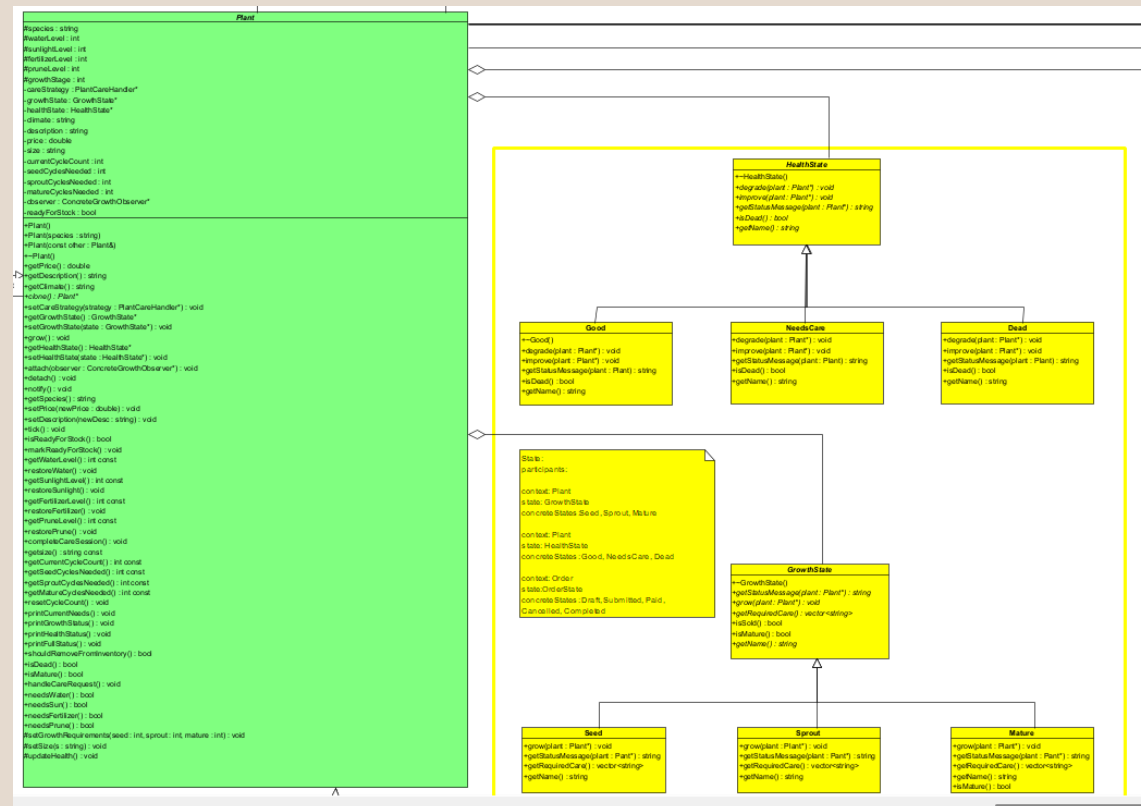
Sprout, Mature

Context:Plant

State: HealthState

concreteStates: Good.

NeedsCare, Dead



Observer

Participants:

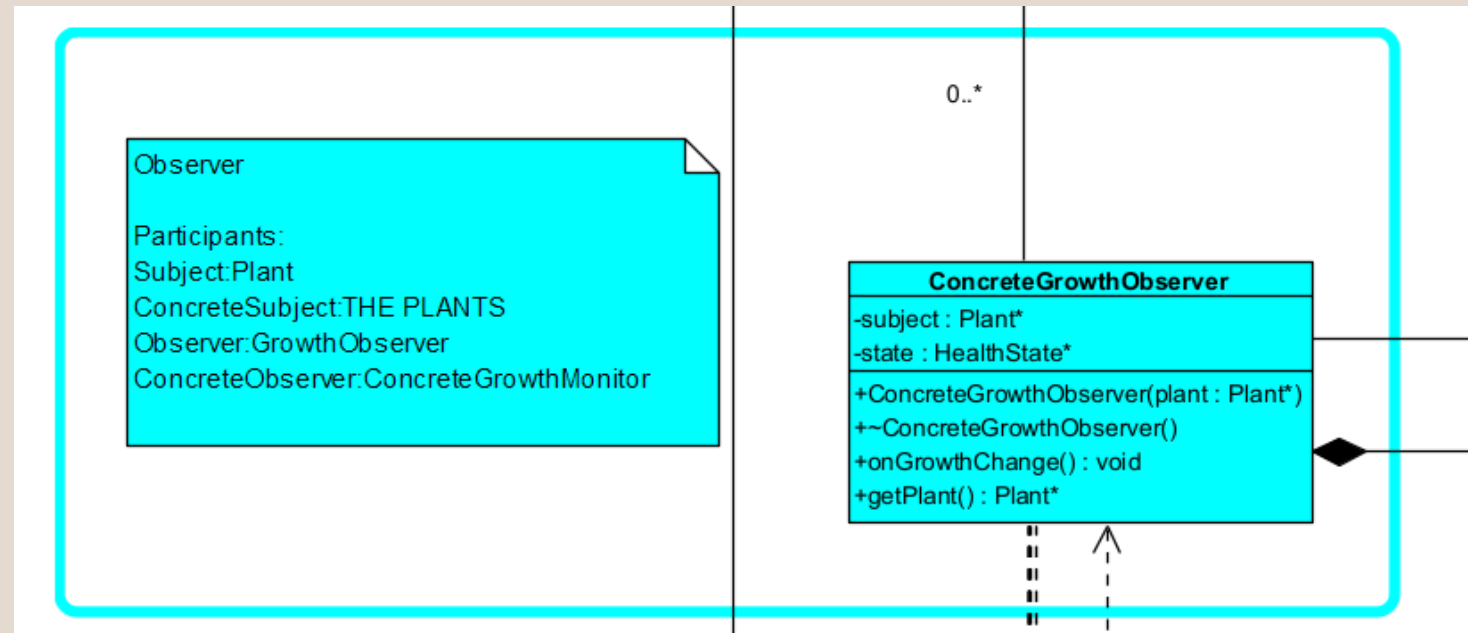
Subject: Plant

ConcreteSubject: The plants

Observer: GrowthObserver

ConcreteObserver:

ConcreteGrowthMonitor



Chain Of Responsibility

Participants:

Handler:

PlantCareHandler

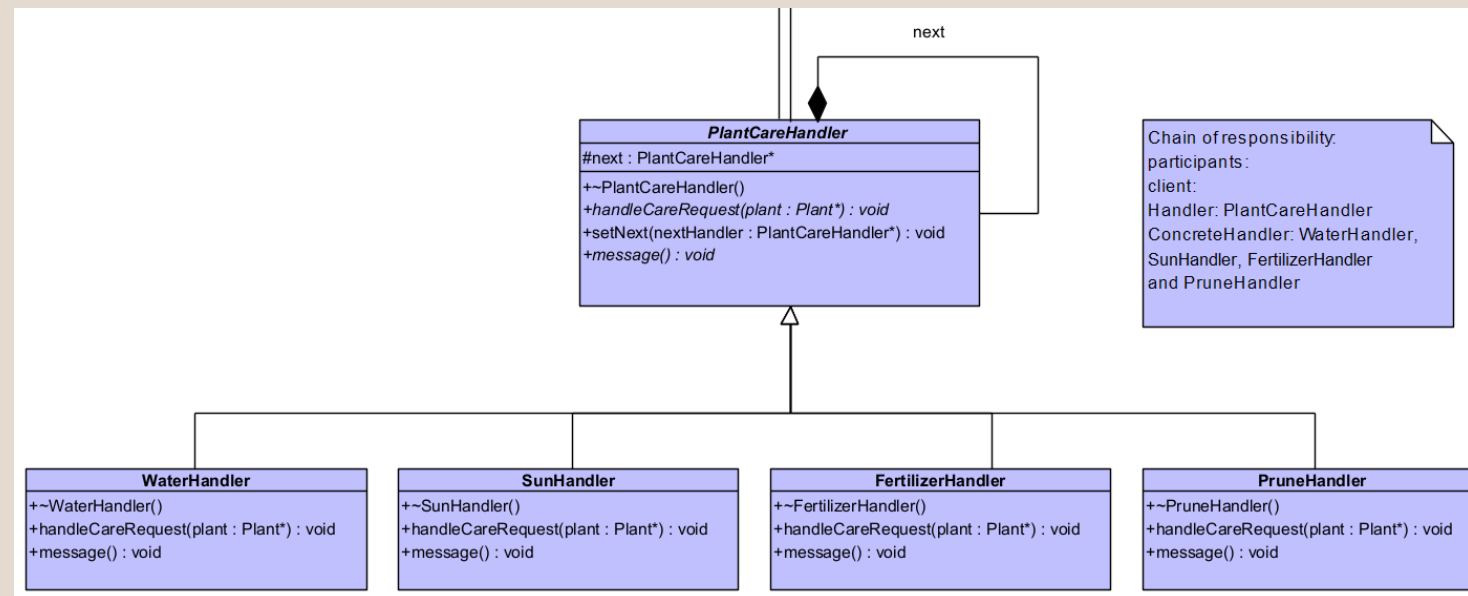
ConcreteHandler:

WaterHandler,

SunHandler,

FertilizerHandler,

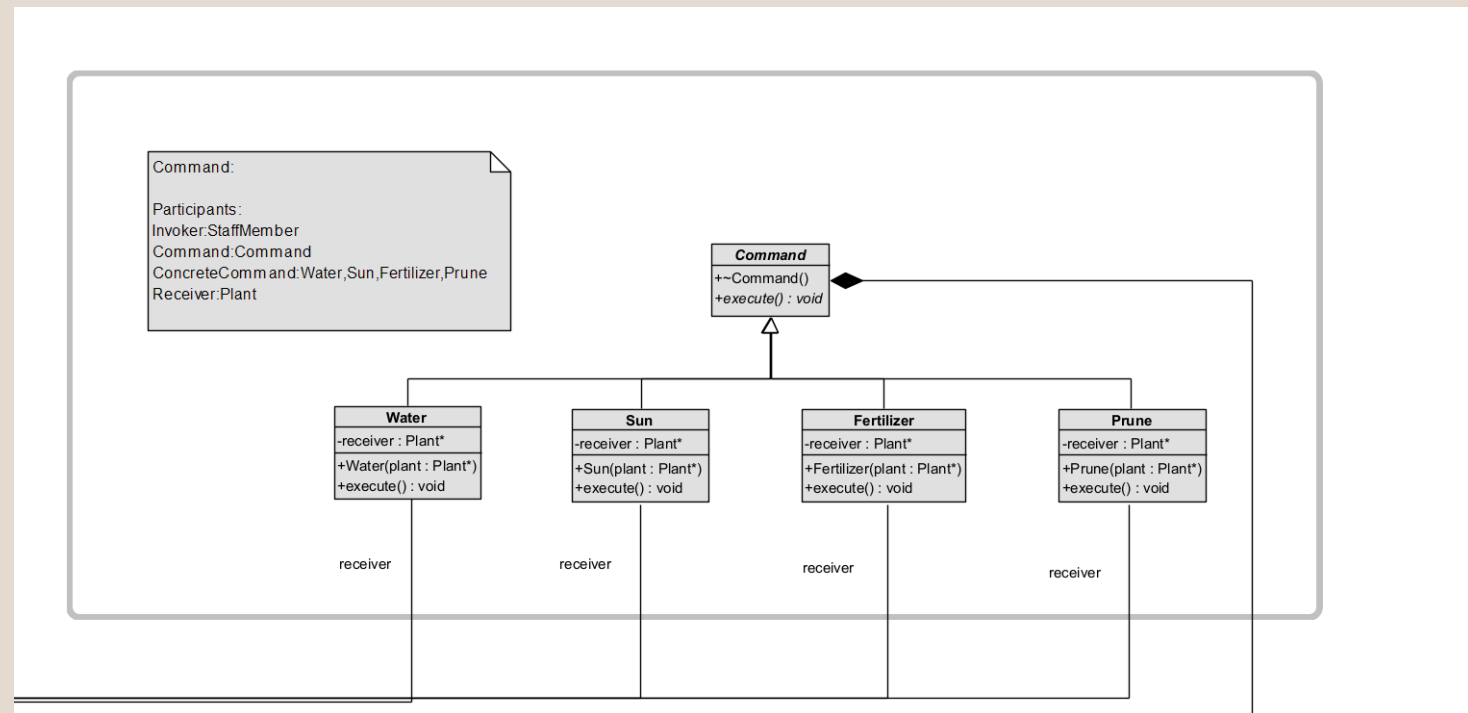
PruneHandler



Command

Participants:

Invoker: StaffMember
Command: Command
ConcreteCommand: Water, Sun, Fertilizer, Prune
Receiver: Plant



Iterator

Participants:

Iterator: Iterator

concreteIterator:

InventoryIterator,

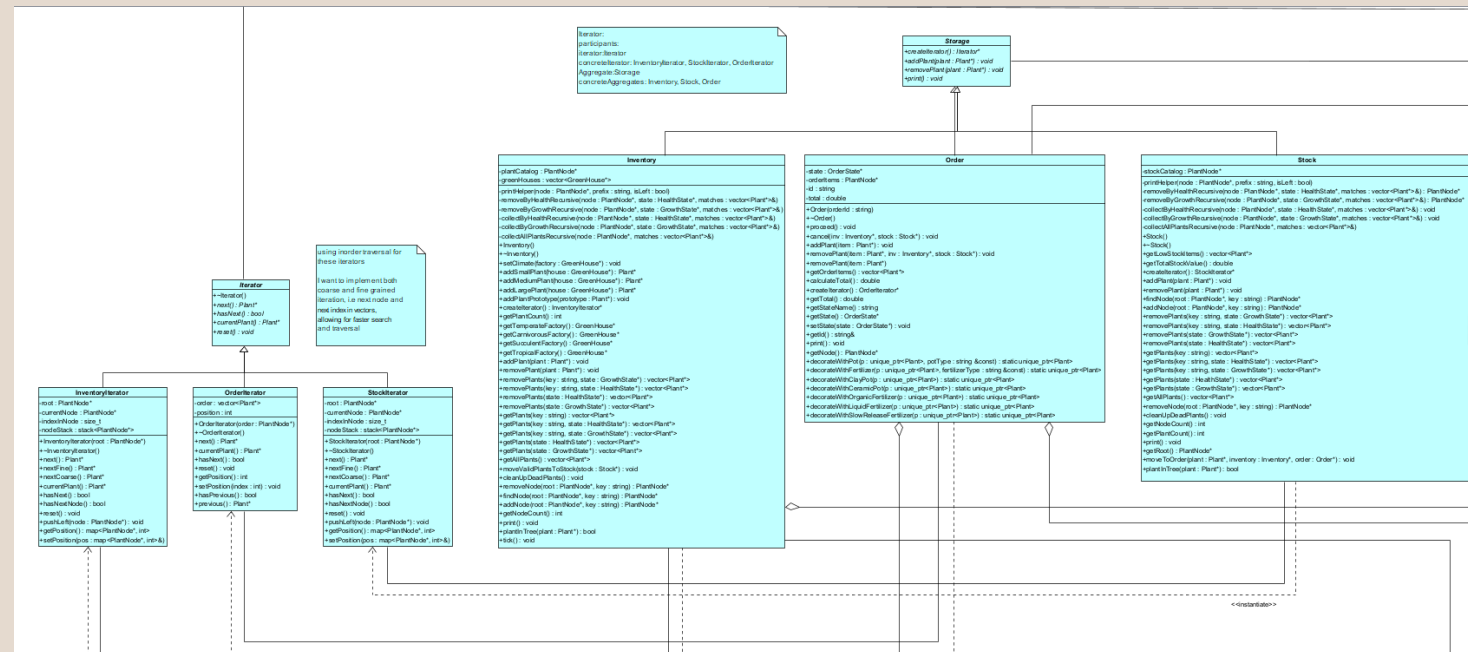
StockIterator,

OrderIterator

Aggregate: Storage

concreteAggregates:

Inventory, Stock, Order



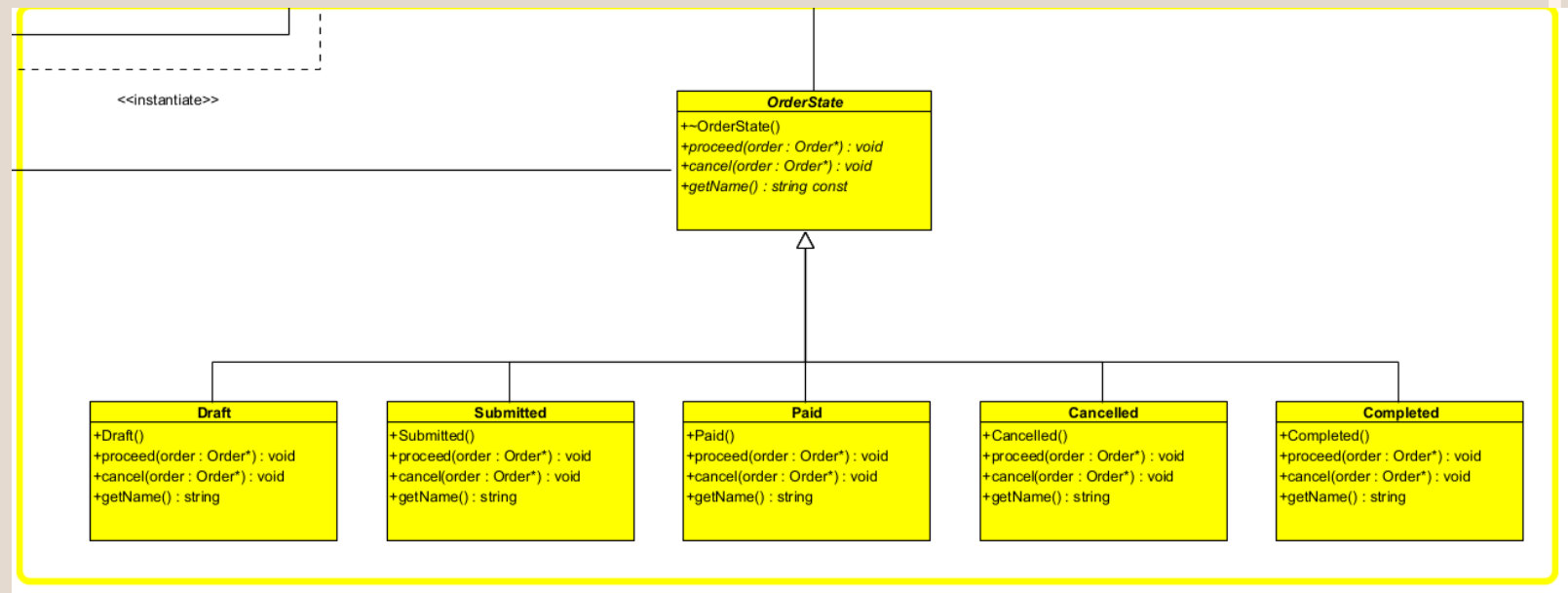
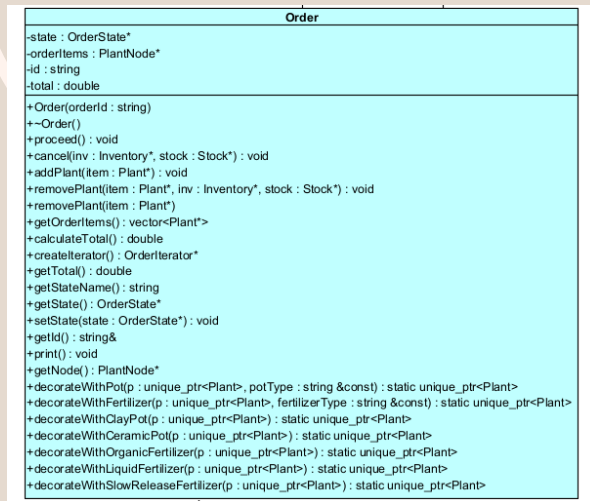
State

Participants:

Context: Order

State: OrderState

concreteStates: Draft,
Submitted, Paid,
Cancelled, Completed



Mediator

Participants:

Mediator: CommMediator

Colleague: StaffMember

ConcreteMediator:

ConcreteCommMediator

ConcreteColleague1: Worker

ConcreteColleague2: Manager

Mediator: GrowthMediator

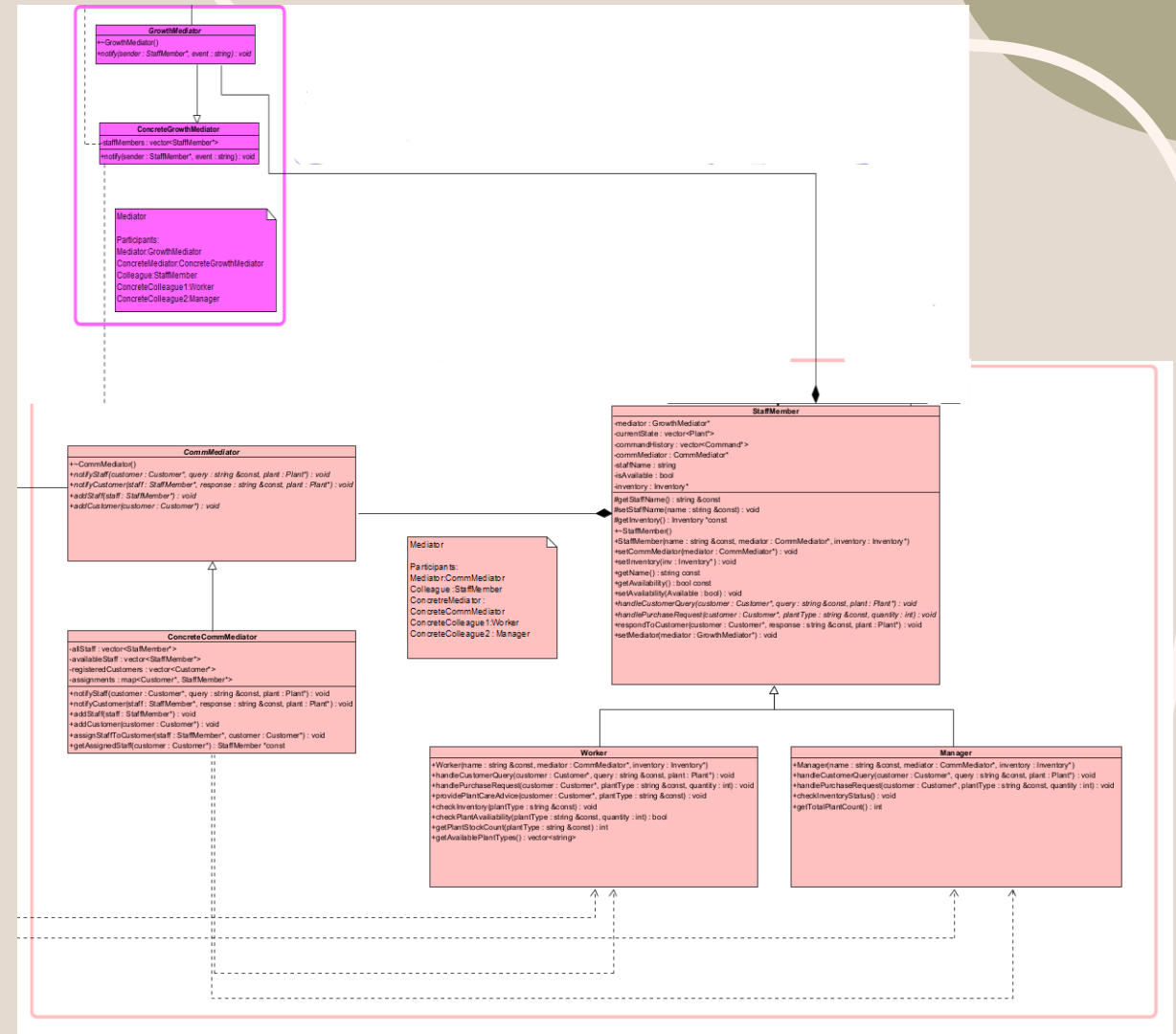
ConcreteMediator:

ConcreteGrowthMediator

Colleague: StaffMember

ConcreteColleague1: Worker

ConcreteColleague2: Manager



Composite

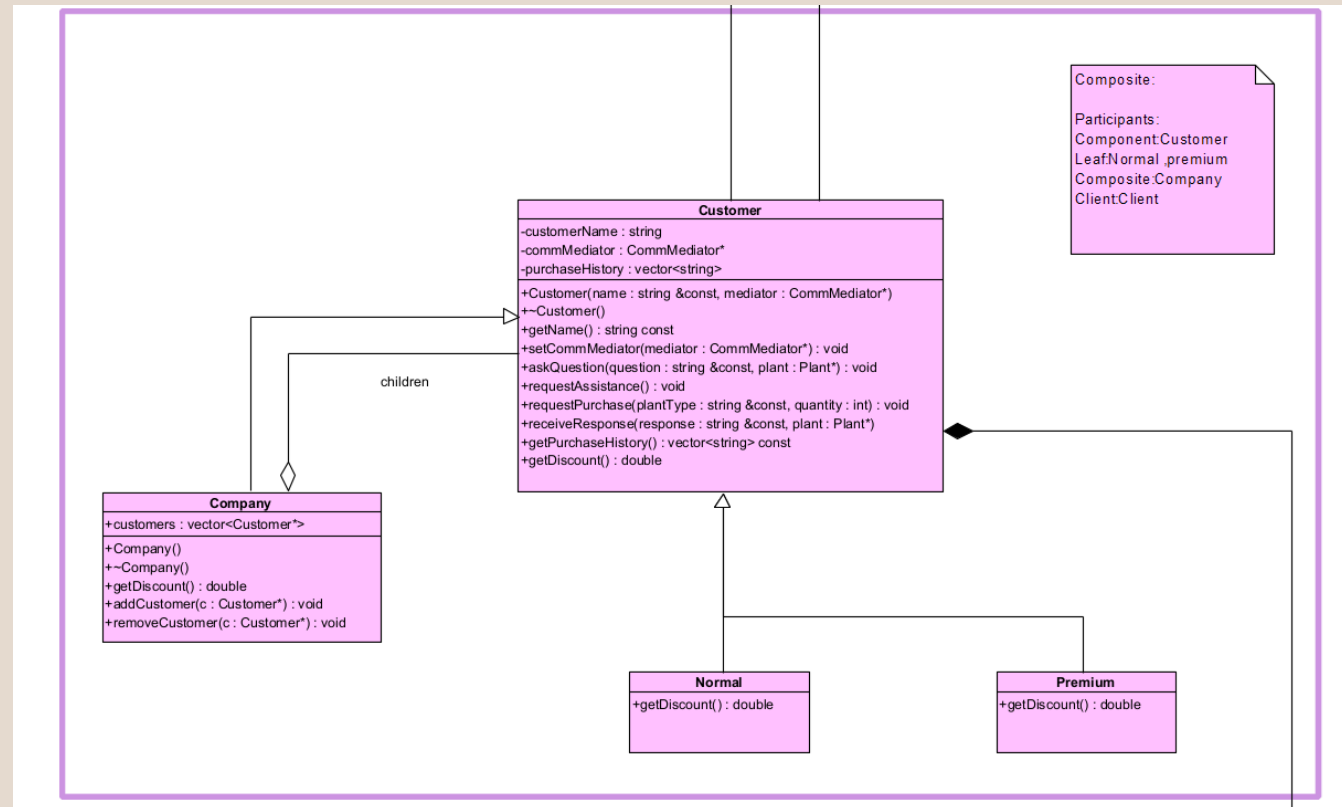
Participants:

Component: Customer

Leaf: Normal, Premium

Composite: Company

Client: Client



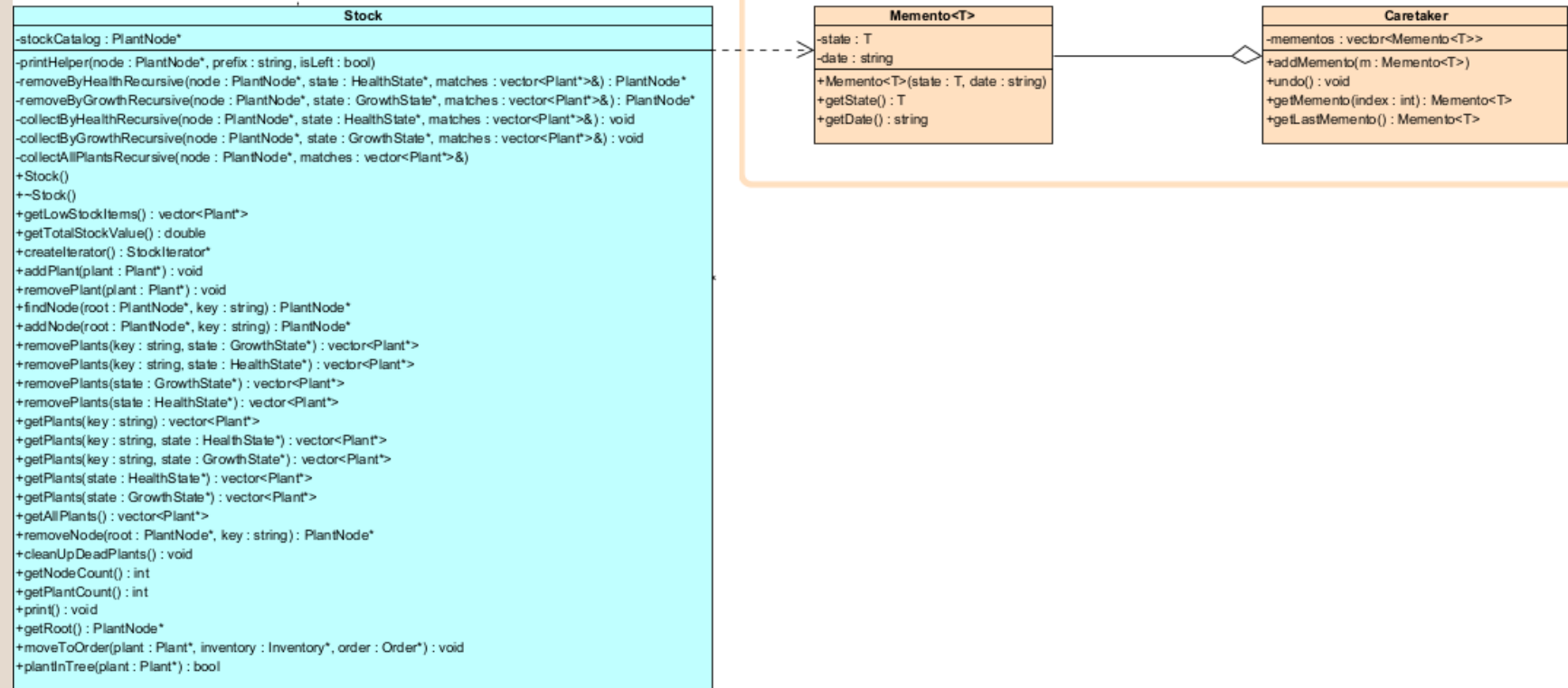
Memento

Participants:

Memento: Memento

CareTaker: Caretaker

Originator: Stock





LETS DEMO

Memory Safety Unit testing

All tests completed successfully!

==13383==

==13383== HEAP SUMMARY:

==13383== in use at exit: 0 bytes in 0 blocks

==13383== total heap usage: 1,123,046 allocs, 1,123,046 frees, 33,304,994 bytes allocated

==13383==

==13383== All heap blocks were freed -- no leaks are possible

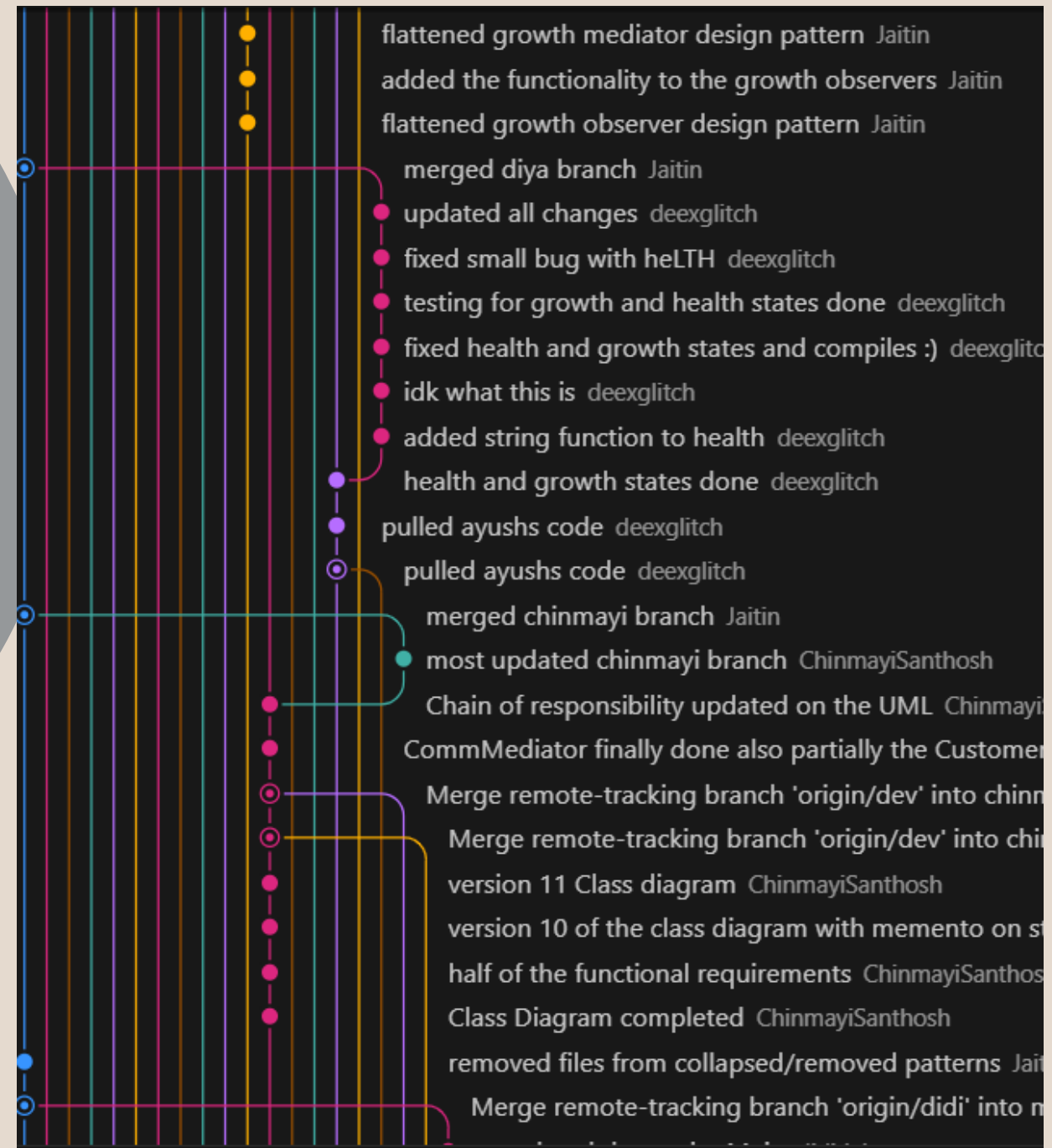
==13383==

==13383== For lists of detected and suppressed errors, rerun with: -s

==13383== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0 from 0)

Git Strategy

We used the gitFlow branching strategy
Makes use of feature branches and multiple primary branches.
We then merged into demo main
And then the final merge into main branch



Commits

WHEN RUNNING COMMAND `GIT SHORTLOG -SN`

NOTE THAT SOME USERNAMES APPEAR TWICE

HENCE OUR FINAL TOTALS ARE:

JAITIN: 56

MAHADIO: 25

DIYA: 27

CHINMAYI:42

FABIO:25

SHAVIR:25

AYUSH: 68

```
diya@DESKTOP-FKIGHQ5:/mnt/c/Users/Diya/OneDrive -  
r2/COS-214-Final-Project$ git shortlog -sn  
68  Ayush-B99  
56  Jaitin  
35  Ayush  
32  Chinmayi  
32  Life220  
25  diditlaka  
22  ShavirV  
17  Diya Narotam  
10  ChinmayiSanthosh  
10  Shavir Vallabh  
10  deexglitch  
1  Wave2055
```


Linting

← Lint Code

Linting #3

Re-run all jobs

Summary

Jobs

Run details

Usage

Workflow file

Triggered via push 43 minutes ago

Status

Total duration

Artifacts

ChinmaySanthosh pushed

cd00b89

main

Success

33s

-

lint.yml

on: push

lint 30s

lint

succeeded 43 minutes ago in 30s

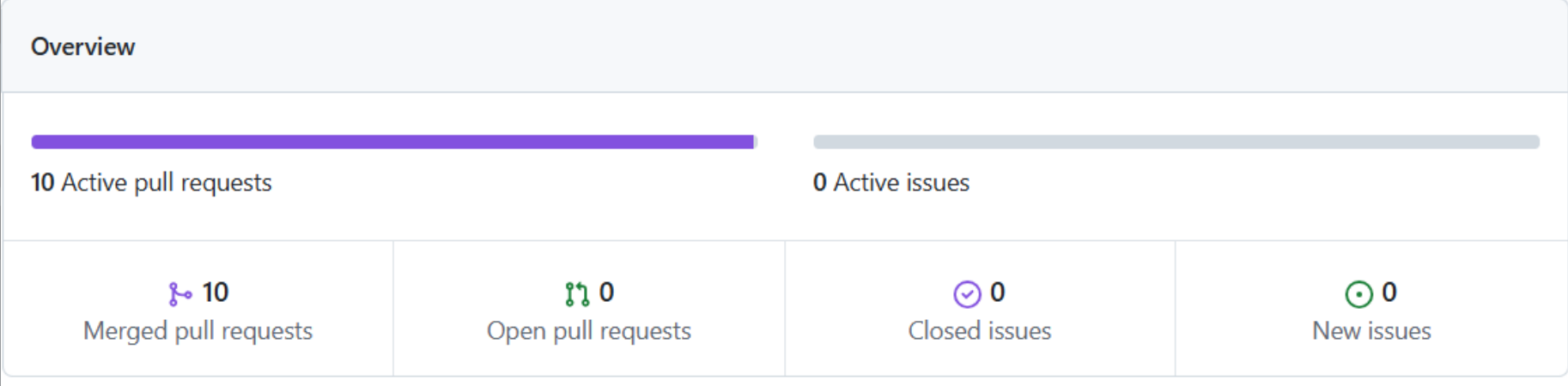
- ✔ Set up job
- ✔ Checkout code
- ✔ Install clang-format
- ✔ List C++ files
- ✔ Check formatting (non-fatal)
- ✔ Show what would change
- ✔ Post Checkout code
- ✔ Complete job

Workflow file for this run

[github/workflows/lint.yml](#) at [cd00b89](#)

```
1 name: Lint Code
2
3 on:
4   push:
5     branches: [ main, develop ]
6   pull_request:
7     branches: [ main ]
8
9 jobs:
10  lint:
11    runs-on: ubuntu-latest
12
13    steps:
14      - name: Checkout code
15        uses: actions/checkout@v4
16
17      - name: Install clang-format
18        run: |
19          sudo apt update
20          sudo apt install -y clang-format
21
22      - name: List C++ files
23        run: |
24          find . -name "*.cpp" -o -name "*.h" -o -name "*.hpp" | while read file; do
25            echo "  $file"
26          done
27
28      - name: Check formatting (non-fatal)
29        run: |
30          if find . -name "*.cpp" -o -name "*.h" -o -name "*.hpp" | grep -q .; then
31            find . -name "*.cpp" -o -name "*.h" -o -name "*.hpp" | xargs clang-format -n --verbose || echo "Formatting issues found (but not failing workflow)"
32          else
33            echo "No C++ files to check"
34          fi
35
36      - name: Show what would change
37        run: |
38          if find . -name "*.cpp" -o -name "*.h" -o -name "*.hpp" | grep -q .; then
39            find . -name "*.cpp" -o -name "*.h" -o -name "*.hpp" | xargs clang-format -i
40            git diff --color || echo "No differences found"
41            # Revert the changes so we don't modify the repo
42            git checkout -- .
43          else
44            echo "No C++ files found"
45          fi
```


Pull requests



ReadMe

COS 214 - Plant Nursery Management System

  **Completed**  **COS 214 Project**

Branch Structure

- `main` - Production-ready code with final implementation
- `dev` - Development branch for ongoing work
- Feature Branches - Individual feature development

Project Overview

This C++ project implements a comprehensive plant nursery management system for COS 214, showcasing object-oriented design principles and software design patterns in C++. It involves a variety of features and interactions, such as being able to view plants, create orders and take care and monitor plant growth and many more.

Project Structure (Main Branch)

UML Diagrams

 README

Structural Diagrams

- [Class Diagram](#) - System class structure and relationships
- [Object Diagram](#) - Runtime object instances and links
- [Communication Diagram](#) - Object interactions and messaging

Behavioral Diagrams

- [Sequence Diagrams](#) - Time-ordered object interactions
- [State Machine Diagram](#) - Object state transitions
- [Activity Diagram](#) - Business process workflows

Team Members & Contributions

Team Member	Role & Focus	Key Contributions	Design Patterns & Features
Ayush Beekum	<i>Structural Foundations & Creational Patterns</i>	- Object Diagram design - Plant creation systems	Abstract Factory: Factory interfaces for plant families Decorator: Flexible plant enhancements Prototype: Efficient plant cloning
Diya Narotam	<i>Plant Lifecycle Management</i>	- Class Diagram design - Plant health & growth systems	State - HealthState: Dynamic transitions (Good → NeedsCare → Dead) State - GrowthState: Growth progression (Seed → Sprout → Mature)
Jaitin Moodally	<i>Request Handling & Command System</i>	- Communication Diagram - Plant care operations	Command Pattern: Encapsulated plant care operations Chain of Responsibility: Sequential request handling Observer Pattern: Monitors plant growth changes
Shavir Vallabh	<i>Data Access & Order Management</i>	- Sequence Diagram - Plant collection traversal	Iterator Pattern: BST-based collection traversal State - OrderState: Order lifecycle (Draft → Completed → Paid → Cancelled)
Fabio Berrino	<i>System Persistence & Demonstration</i>	- Activity Diagram - System state management	Memento Pattern: Templatized state capture/restore DemoMain: Comprehensive system demonstration
Chinmayi Santosh	<i>Communication Mediation</i>	- Class Diagram design - Object communication	Mediator Pattern: Centralized worker-customer communications
Mahadio Tiaka	<i>Hierarchical Structures</i>	- State Machine Diagram - Customer management	Composite Pattern: Customer grouping hierarchy

Doxygen

File List	
Here is a list of all documented files with brief descriptions:	
[detail level 1 2]	
▼ include	
AloeVera.h	Concrete product representing an Aloe Vera succulent plant (medium size)
BirdOfParadise.h	Concrete product representing a Bird of Paradise tropical plant (medium size)
Cancelled.h	Concrete OrderState representing an order that has been cancelled
Caretaker.h	Implements the Caretaker role in the Memento design pattern
CarnivorousPlantFactory.h	Concrete factory for creating carnivorous plants
Client.h	
Command.h	Abstract command interface for plant care operations
CommMediator.h	Abstract mediator interface for customer-staff communication
Company.h	Composite component representing corporate customer groups
Completed.h	Concrete OrderState representing an order that has been completed
ConcreteCommMediator.h	Concrete implementation of the communication mediator
ConcreteGrowthObserver.h	Observer that monitors plant growth changes and coordinates care
Condelabra.h	Concrete product representing a Condelabra succulent plant (large size)
Customer.h	Abstract component in the Composite pattern for customer hierarchy
Daisy.h	Concrete product representing a Daisy temperate plant (small size)
Dead.h	Concrete state representing a deceased plant
Draft.h	Concrete OrderState representing an order in draft form
Fertilizer.h	Concrete command for fertilizing plants
FertilizerDecorator.h	Concrete decorator for adding fertilizer features to plants
FertilizerHandler.h	Concrete handler for fertilizer-related plant care requests
Good.h	Concrete state representing a healthy plant
GreenHouse.h	Abstract factory for creating plants of different sizes

Questions,
comments &
concerns





thank you

MAHADIO

SHAVIR

DIYA

AYUSH

CHINMAYI

JAITIN

FABIO

